

Increase I/O bound tensorflow training speed

Asked 4 years, 3 months ago Modified 4 years, 3 months ago Viewed 1k times



3



I am facing a problem of improving the training speed / efficiency of a Tensorflow implementation of point cloud object detection algorithm.

The input data is a [8000, 100, 9] float32 tensor, with a size roughly 27MB per sample. On a batch size of 5, data loading becomes a bottleneck in training as most of the time GPU utilization rate is 0% until data arrives.

I have tried the following methods to increase data loading speed.

1. Use `num_parallel_calls` in `tf.Dataset .map` API, and use multiple threads for reading this big tensor. The problem is `.map` wraps a `py_func` which is subject to Global Interpreter Lock and thus multi-threading does not improve I/O efficiency.
2. Use `tf.Dataset .interleave` API. Since it's also multi-threading based, it has the same problem as 2.
3. Use TFRecord format. This is even slower than method 1 and 2. Possibility is TFRecord will convert tensor to numpy, then serialize numpy to bytes, then wrap this bytes to tensorflow structure and write to disk. Numpy to Tensor takes a long time for my data as measured by `tf.convert_to_tensor()`.

Any suggestions how to move forward would be helpful. Thanks!

Follow up on comments

1. Am I using slow disks? Data is stored on a mounted disk. Could be a reason.
2. Can the data be fit into GPU memory? Unfortunately no. There are ~70,000 samples. I tried cache a small dataset into RAM and GPU utilization rate is 30%~40%, which is probably the highest expectation for this particular network.

`python` `tensorflow` [Edit tags](#)

Share Edit Follow Close Flag

edited Jun 3, 2020 at 19:00



user4157124

2,908 14 29 43

asked Jun 3, 2020 at 16:50



yuqli

5,031 8 32 50

- 1 I assume you are reading this data off disk. Are you using slow disks? Follow up question, how big is the actual dataset? Because if it fits on your GPUs you could use the Rapids library, read it onto the GPU and do the processing entirely on GPU. I don't know what sort of hardware you have access to, but the library was designed for throughput problems like these. Especially if you have access to a GPU grid. – [DejaVuSansMono](#) Jun 3, 2020 at 16:52



Apache spark could be useful for this purpose, towardsdatascience.com/... – [deepak sen](#) Jun 3, 2020 at 18:01

1 Answer

Sorted by: [Reset to default](#) [Date modified \(newest first\)](#)



3



Some ideas:

1. You should use a combination of 1,2 and 3. If you save your files as `TFRecords`, you can read them in parallel, that's what they are designed for. Then, you will be able to use `num_parallel_calls` and `interleave`, because that way you don't have to wrap a `py_func`.
2. `.map` doesn't have to wrap a `py_func`, you could for example use `tf.keras.utils.get_file`. That way you also avoid using `py_func` and use `num_parallel_calls` efficiently. I still recommend using `TFRecords`, they are designed for this use case.
3. Another option is to use an SSD to store your data instead of a Hard Disk.
4. You can also look into the `.cache` function of the `tf.Dataset` API. Maybe you can try loading a random subset of the data, training multiple epochs on that, and then in the mean time fetch another subset of the data (using `tf.prefetch`),

9/10/24, 12:22 AM

python - Increase I/O bound tensorflow training speed - Stack Overflow

and then train multiple epochs on that, and so on. This idea is more of a long shot as it might affect performance, but it just might work in your case.

Share Edit Follow Flag

answered Jun 3, 2020 at 17:52



[Frederik Bode](#)

2,734 1 12 18